MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Kevin G. Vega
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre

Panel Member

Date signed: ______________

Michelle Elija B. Santos

Panel Member

Date signed: ______________

Ian Peter L. Lastimosa

Panel Member

Date signed: ______________

Kevin G. Vega

Project Advisor

Date signed: ______________

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT

Chair, Department of Computer Science

Date signed: ____________________

Joshua C. Martinez MIT

Dean, College of Computer Studies

Date signed: ____________________

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Kevin G. Vega
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Kevin G. Vega

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ leverages a Deep Recurrent Q-Network (DRQN) architecture and league-based self-play training to address the large state space of Stratego and hidden information challenges. The architecture integrates an Attention as Recurrent Neural Network (AAREN) to maintain refined belief states over long game histories, enabling opponent modeling and strategic value contextualization. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating Probabilistic Belief States (PBS) and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

1.1 Project Context

Games have a history of being a metric for how Artificial Intelligence (AI) progresses and performs in the current time [40]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is a feature found in many multiplayer games known as the "fog of war" [53], where the game environment is not fully revealed, which makes it an example of an imperfect information game [28]. Imperfect information games create a scenario where players develop unconventional strategies without having full knowledge of the game's environmental state [53]. Solving these kinds of problems and determining the optimal strategy remains a significant challenge for Artificial Intelligence [42].

This study focuses on mastering imperfect information games through Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [16, 12]. With DQN, the agents can utilize deep neural networks to approximate the Q-function that maps state-action pairs for the expected reward. This study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework, which extends the basic DQN architecture with Rainbow enhancements including distributional value estimation and noisy exploration networks [21]. The framework integrates an AAREN (Attention as Recurrent Neural Network) module [19] for processing extended observation sequences and maintaining a Probabilistic Belief State (PBS) that tracks uncertainty about hidden opponent pieces. League-based self-play training [44] ensures that agents develop robust strategies by competing against diverse historical opponents. This approach is suited well for Stratego, where agents are trained to compete against each other, developing strategies through repeated experience while reasoning under uncertainty about the opponent's setup and the environment's hidden state.

1.2 Purpose and Description

This research aims to evaluate whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can be used to develop intelligent agents capable of mastering imperfect information games. By focusing on the limited information available to players in environments such as the board game Stratego, this study seeks to address the challenge of making decisions under uncertainty while incorporating strategic planning. Although existing tools for Stratego provide some analytical support, they are often limited in fostering deeper reasoning about the game's tactical complexities.

To advance beyond these limitations, this research seeks to provide an algorithmic foundation for future analytical systems while also emphasizing the development of critical thinking in solving imperfect information games. By modeling adaptive strategies and reasoning under uncertainty, the study highlights how artificial intelligence can mirror and enhance human-like logical reasoning, offering valuable insights for both game analysis and broader decision-making domains.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How does the MARQ framework perform in terms of convergence speed and win-rate against heuristic baselines in a partially observable environment?

2. Does the MARQ framework demonstrate statistically significant superiority over standard DQN and rule-based agents across varying opponent difficulty tiers?

3. What is the impact of integrating the AAREN module on the agent's ability to infer hidden opponent piece configurations compared to approaches without explicit belief state modeling?

1.3 Objectives

The main objective of this study is to develop and evaluate Multi-Agent Reinforcement Learning with Deep Q-Networks for learning appropriate strategies in an imperfect information game, where the agents operate under hidden state variables to improve decision-making, adaptability, and strategic reasoning against both human and AI opponents. This can be achieved through the following:

- Introduction of Stratego to the community
- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, Stratego being the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) using AAREN, and the training and optimization of the AI agent to adapt to differences in game states. The evaluation of the agent's performance would primarily be on Stratego's environment. The trainings are limited to the personal devices of the researchers and would not rely on cloud-based computing resources. Since the AI is going to learn alongside the community the player and AI head-to-head will be analyzed mostly qualitative. The goal will be achieving the same human critical thinking in the model.

1.5 Significance of the Study

The development of the MARQ framework addresses limitations in current imperfect information game AI, where existing systems either require large computational resources or employ model-free approaches that limit strategic depth. The MARQ framework's significance lies in its explicit opponent modeling through AAREN and probabilistic belief states, and hybrid DQN architecture that balances strategic depth with computational tractability. The synchronized learning evaluation framework provides insights into AI adaptability against improving human opponents, with implications extending beyond gaming to domains characterized by incomplete information and strategic interaction, including cybersecurity, financial trading, and strategic planning.

1.6 Definition of Terms

AAREN (Attention as Recurrent Neural Network)

A neural architecture that combines the parallel training efficiency of Transformers with the sequential processing of RNNs. In the MARQ framework, AAREN processes observation sequences to maintain and update the Probabilistic Belief State.

Counterfactual Regret Minimization (CFR)

An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)

A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Distributional Reinforcement Learning

An approach that models the full distribution of returns rather than just the expected value. The MARQ framework uses C51 (Categorical DQN) with 51 atoms to capture uncertainty in value estimates.

Fog of War

A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Information Set Monte Carlo Tree Search (IS-MCTS)

A search algorithm for imperfect information games that samples consistent game states from belief distributions and performs tree search on information sets rather than individual states.

League Training

A self-play training paradigm where agents compete against a diverse pool of historical opponents, preventing strategy cycling and ensuring robust generalization across different playing styles.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

Probabilistic Belief State (PBS)

A probability distribution over possible opponent piece configurations, maintained and updated by the AAREN module as observations reveal information during gameplay.

Rainbow DQN

An enhanced DQN architecture that combines multiple improvements including distributional RL, noisy networks for exploration, and dueling network architecture for improved stability and sample efficiency.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [40, 16]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [14]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [40, 22]. The case is the same for both perfect information and imperfect information games [40]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [34]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [39]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [35, 8]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [56]. Continuing development of AI algorithms in imperfect information games advances our understanding of decision-making under uncertainty and promotes strategic planning.

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [40]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in Starcraft II, which is a real-time strategy video game, unlike SoG, which has not been tested in video games [25].

2.1.2 Deep Q-Networks

One of the most important deep-learning algorithms is called Deep Q-Networks [23]. This algorithm is a type of reinforcement learning algorithm that combines Q-Learning with deep neural networks [9]. A study has been made comparing DQN with Proximal Policy Optimization(PPO) in VizDoom, a 3D environment for reinforcement learning based on Doom, a first-person-based shooter [51]. In the study, the environment would be in a gamemode called DeathMatch, where the main goal is to kill as many bots while minimizing deaths. In PPO with object detection, it learned a policy to run around and collect survival items like medkits and armor rather than focusing on kills, making the bot have close to zero kills. While the bot using DQN with object detection would go for both kills and collecting survival items, making the kill-death ratio higher than PPO. In short, PPO with object detection focuses on survival by collecting survival items and avoiding combat, while DQN with object detection has a balance of collecting survival items and combat.

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [16]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-

Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [16, 12]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [12]. For scalability, as more agents are being used, the computational power rises, making it expensive [31, 12]. Recent work on Ataraxos demonstrates that these challenges can be addressed through careful regularization: by constraining policy updates toward both exploratory (uniform) and stable (target network) anchors with time-varying coefficients, agents can achieve superhuman performance in complex imperfect-information games while maintaining training stability [43].

2.1.4 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [31, 50]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [31]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [49].

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [35].

2.2.1 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.2 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [19]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

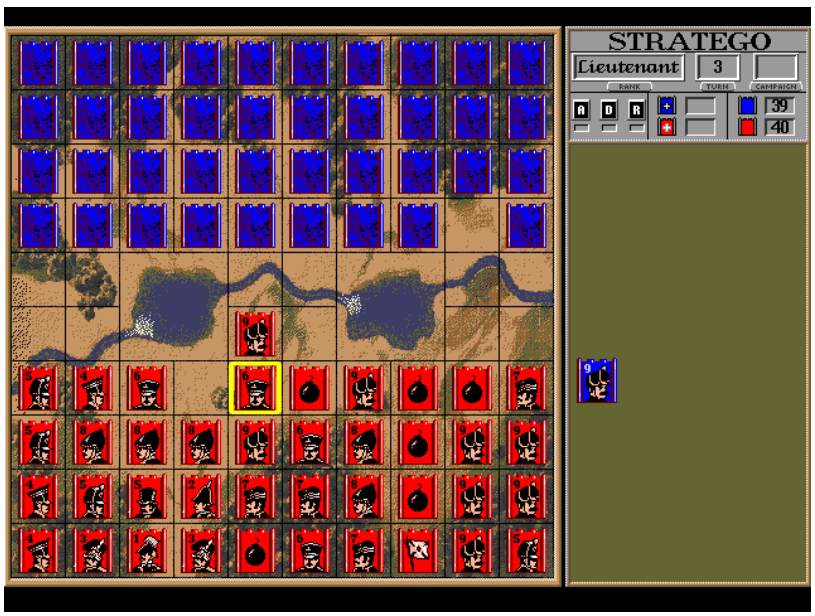

Figure 2.1: Stratego Game board.

2.4 Board Games in Community Building

Modern board games are becoming more widespread, expanding their market share each year. During COVID-19 lockdowns, board game sales increased because people were stuck at home with family and had free time. Also, playing board games during that time helped with stress, isolation, and anxiety. Certain games like Pandemic, a cooperative strategy board game where players work together to stop global disease outbreaks by discovering cures before time runs out, became more popular due to their reflection of the real-world scenario. Board game communities differ between the West and the East [20]. From the east, board games are considered social activities, while in the west, board games grew as hobbies, communities focusing on the gameplay and mechanics. In Hong Kong, there are two distinct board game communities named BGHK and Oasis. BGHK consists of Cantonese-speaking individuals who treat board games as social bonding. They even event sessions for "party game days" and "welcome newbies," emphasizing making friends rather than the game itself. While the Oasis group focuses on the hobby, playing newly released and complex games. Attracting players with the mechanics and novelty rather than just socializing. A study has been made to use board games as an answer to the problem of difficulty in creating a sense of "togetherness or kinship," and board games have advantages for improving the cognitive function of older people [3]. In the study, the Tresna Werdha Budi Pertiwi Social Home has a problem of having a sense of family among residents. By playing together, the elderly had consistent interaction with each other, reducing the feeling of loneliness and detachment. Board games not only provide entertainment but also memory stimulation and improve social bonds. Board games provide a face-to-face interaction requiring people to sit together at a table, unlike in video games, where players are often separate from each other [37]. Every shared experience creates a unique microenvironment where players enter a separate reality governed by the game's rules. In the space, the players share emotions and strategy. Sharing these emotions helps strengthen friendships, family bonds, and trust. Many board game players first connect through gaming with friends and family, which can branch out to dedicated groups or conventions, and expand their social circle.

2.5 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [33]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [55].

2.6 Application to Other Game Environments

Although this study focuses on Stratego as the game environment, the MARQ framework can also be applied to other strategic and simulation-based games that involve decision-making and hidden information. Sports management titles such as Madden NFL's Dynasty Mode, NBA 2K's franchise mode, Esports Manager, and Football Manager feature complex systems of player development, transfer, and long-term planning, where agents must act without complete information about the future player performance and market behavior. In these environments, the MARQ framework could model AI agents capable of learning optimal trade strategies or budget allocations through reinforcement learning. The framework's belief-state modeling would allow the AI to check hidden player attributes or opponent strategies, while the Deep Q-Network could optimize sequential decisions across multiple simulated seasons.

Similarly, the framework could be applied to a game called Clash Royale, where players must make fast tactical decisions without full knowledge of their opponent's hand or resource level. The Probabilistic Belief State in this setting could estimate the likelihood of specific opponent cards being available, while the Deep Q-Network would select optimal actions for troop deployment and timing. IS-MCTS could be used to search over possible opponent hands, enabling informed decision-making for lane control or counter-attack scenarios.

Chapter 3

Theoretical Frameworks

3.1 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961 [4]. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. Victory is achieved by capturing the opponent's Flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

The theoretical state space of Stratego demonstrates the computational challenge:

$$|\mathcal{S}| = \binom{40}{1,1,2,3,4,4,4,5,8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.1}$$

This state space complexity estimate was established by Perolat et al. [36]. This high complexity necessitates knowledge-limited reasoning and belief state management rather than exhaustive

Figure 3.1: Stratego Pieces

enumeration of possible states.

3.2 MARQ Framework Overview

The MARQ framework, a Multi-Agent Rainbow Deep Q-Networks system, employs self-play techniques and deep reinforcement learning to train agents capable of handling Stratego's large game state space and imperfect information [8]. The framework uses a league-based self-play system where agents learn by competing against a diverse pool of opponents, including their own previous versions and exploratory variants.

This approach addresses the fundamental challenge of Stratego: reasoning about hidden information without computing intractable common-knowledge sets. While the number of possible game states exceeds 10^{535} and initial setups alone reach 10^{66} per player [36], the MARQ framework manages this complexity through belief state management, probabilistic reasoning, and strategic pattern recognition. The framework combines deep reinforcement learning for value estimation, attention mechanisms for processing game history, and controlled strategy mixing to prevent exploitation, while maintaining computational tractability even in this vast imperfect-information domain.

Figure 3.2: Stratego Game board.

The MARQ framework operates through the hierarchical integration of its components. The system starts with observation processing, where the AAREN module ingests raw game positions to construct and maintain a probabilistic belief state (PBS). This belief state forms the base of information used for all subsequent reasoning. The entire architecture is embedded within a league-based training paradigm involving simultaneous training and evaluation of MARQ agents.

3.3 MARQ Policy Definition

Policy optimization in reinforcement learning refers to the iterative refinement of an agent's decision-making strategy to maximize cumulative expected reward [45]. In multi-agent environments characterized by imperfect information, this optimization process must produce policies that are not only effective against static opponents but also robust to the inherent uncertainty of hidden game states and the adaptive behaviors of learning adversaries. The MARQ framework addresses these challenges through a hierarchical policy architecture that integrates belief state reasoning with value-based decision making.

The policy learning process in MARQ operates across multiple interconnected objectives. The

Table 3.1: Key Symbols and Descriptions from the MARQ Framework

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Belief state at time t representing probability distributions over opponent configurations	Probability distribution over possible identities and positions of hidden opponent pieces
$v_i^\pi(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief state $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	PBS evaluator network output (quality score)	Confidence score for belief distribution accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

primary objective concerns game outcomes, where the agent learns to maximize win probability through strategic piece deployment and tactical combat decisions. A secondary objective involves belief state accuracy, where the Probabilistic Belief State maintained by the AAREN module is continuously refined through comparison with revealed ground truth positions during gameplay. These objectives are jointly optimized, as accurate belief states enable better decision-making, while strategic decisions that reveal opponent information improve future belief quality.

3.3.1 Core Policy Framework

A policy in the MARQ framework, denoted as $\pi_i(a|I_i)$, defines the probability distribution over actions for agent i conditioned on its current information set I_i. Unlike policies in perfect information games that operate on complete game states, MARQ policies must reason under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent, including visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The fundamental challenge in imperfect information games lies in the requirement that agents make decisions without access to the opponent's private information. In Stratego, this private information includes the identity and position of all opponent pieces that have not yet been revealed through combat. The policy must therefore integrate uncertain beliefs about opponent configurations into its decision process, selecting actions that perform well across the range of possible hidden states.

To address this challenge, the MARQ policy operates on belief states $\mathcal{B}_t$ rather than attempting to enumerate all consistent game states. The belief state represents a probability distribution over possible opponent piece configurations, maintained and updated by the AAREN module as new observations become available. The policy is thus defined as:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{belief state } \mathcal{B}_t) \tag{3.2}$$

This formulation captures the key insight that optimal play in imperfect information games requires probabilistic reasoning over hidden states. The policy learns to weight actions based on their expected value across the belief distribution, rather than optimizing for any single assumed opponent configuration.

3.3.2 Belief State Optimization

The quality of the Probabilistic Belief State directly impacts decision quality, as inaccurate beliefs lead to poorly calibrated action values. MARQ therefore incorporates belief accuracy as an auxiliary training objective, using revealed piece identities as supervised labels for belief refinement.

When combat reveals the true identity of opponent pieces, the prediction error between the prior belief and the revealed ground truth is measured using Kullback-Leibler divergence:

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}} \left(\text{PBS}_t(p) \| \delta_{s_p^*} \right) \tag{3.3}$$

Here, $\delta_{s_p^*}$ denotes the Dirac delta distribution concentrated on the true piece type s_p^* at the moment of revelation. This error quantifies how surprised the belief system was by the revealed information, with larger values indicating greater divergence between predicted and actual piece identities.

This prediction error is transformed into a belief accuracy reward signal that encourages the policy to develop beliefs that better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = - \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{PBS}_t(p = s_p^*) > 0 \right] \cdot \log \text{PBS}_t(p = s_p^*) \tag{3.4}$$

The optimal policy is then defined as the policy that maximizes the joint objective of game outcomes and belief accuracy:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i} \left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t) \right) \right] \tag{3.5}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the relative importance of belief accuracy versus immediate game rewards.

3.3.3 Bellman Equation with Belief States

The value function in MARQ extends the standard Bellman equation to incorporate belief state evolution. The recursive value decomposition expresses the expected cumulative reward as a function of the current information set and belief state:

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi \left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t \right] \tag{3.6}$$

The reward function decomposes into three components that capture different aspects of strategic value. The capture reward R^{capture} reflects the immediate material consequences of combat, weighted by the relative values of pieces captured and lost. The information reward R^{info} quantifies the strategic value of revealing opponent pieces. The positional reward R^{position} measures territorial control and piece mobility.

3.3.4 Nash Equilibrium in Imperfect Information

The strategic objective of MARQ training is to discover policies that approximate Nash equilibrium strategies [11]. A Nash equilibrium represents a strategy profile where no player can improve their expected payoff through unilateral deviation:

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \tag{3.7}$$

Finding exact Nash equilibria in large imperfect information games is computationally intractable, so MARQ instead seeks approximate equilibria through self-play training with diverse opponent populations.

3.4 Deep Q-Network Architecture

Deep Q-Networks in the MARQ framework operate on belief states rather than complete game states [45]. The network approximates Q-values for probability distributions over opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

The system uses a standard replay buffer for experience storage, sampling transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ uniformly to train the value function.

3.4.1 Rainbow DQN Components

The implementation incorporates key enhancements from the Rainbow DQN architecture [21] to improve stability and sample efficiency.

Distributional RL (Categorical DQN). Stratego's inherent uncertainty leads to multimodal value distributions [7]. The value distribution $Z(s, a)$ is modeled directly rather than just its expectation. The distribution is approximated by a discrete distribution over $N = 51$ atoms $\{z_i\}_{i=1}^{N}$, with support ranging from V_{min} to V_{max}:

$$Z_\theta(s, a) = \sum_{i=1}^{N} p_i(s, a)\delta_{z_i} \tag{3.8}$$

The training objective minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}} Z_{\theta'} || Z_\theta) \tag{3.9}$$

where Φ is the projection operator mapping the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration. To ensure consistent exploration without the need for discrete ϵ-greedy schedules, the framework employs Noisy Linear layers. The weights and biases are perturbed by a deterministic function of factorized Gaussian noise:

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.10}$$

where ϵ_w, ϵ_b are random variables. This allows the network to learn the degree of exploration required for different states.

Dueling Network Architecture. The network explicitly separates the estimation of state values $V(s)$ and advantage values $A(s, a)$ using two separate streams that share a common convolutional feature extractor:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{3.11}$$

This decomposition leads to better policy evaluation in the presence of many similar-valued actions.

3.4.2 Exploration-Driven Q-Value Adjustment

The Q-values are augmented with an exploration bonus based on the uncertainty of the belief state, utilizing an optimism in the face of uncertainty approach:

$$Q_{augmented}(s, a) = Q_{base}(s, a) + \lambda \cdot U(a) \tag{3.12}$$

where $U(a)$ represents the entropy of the belief distribution at the target location, encouraging the agent to investigate unknown opponent pieces.

3.4.3 Training Stability via Regularization

Recent advances in imperfect-information game solving, exemplified by the Ataraxos system which achieved superhuman performance in no-press Diplomacy, demonstrate that training stability is as critical as algorithmic innovation [43]. Deep reinforcement learning agents often struggle with two related problems: sample inefficiency, where most training experiences provide little learning signal, and strategy cycling, where the policy oscillates between different approaches without settling on a robust solution. The MARQ framework incorporates two complementary regularization techniques inspired by this work.

Advantage Filtering for Sample Efficiency. Advantage filtering techniques have been shown to improve sample efficiency in imperfect-information game training [47]. In value-based reinforcement learning, agents learn by comparing their predicted values with observed outcomes. The temporal-difference (TD) error quantifies this discrepancy. Standard experience replay samples transitions uniformly from a buffer of past experiences, treating all transitions equally even though some carry more learning potential. Advantage filtering focuses training on high-error transitions by oversampling the replay buffer and retaining only the top quartile by TD error magnitude:

$$\mathcal{B}_{filtered} = \text{Top}_{n/k}\left(\mathcal{B}_{sampled}, |\delta_i|\right) \tag{3.13}$$

where $\delta_i = r_i + \gamma^n V(s_i') - V(s_i)$ is the n-step TD error for transition i.

Dynamic Damping with Power-Law Annealing. To address strategy cycling, the loss function incorporates two regularization terms based on Kullback-Leibler divergence:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \alpha(t) \cdot D_{KL}(\pi \| \pi_{uniform}) + \beta(t) \cdot D_{KL}(\pi_{target} \| \pi) \tag{3.14}$$

The coefficients follow power-law schedules with opposing dynamics:

$$\alpha(t) = \alpha_0 \cdot (1 - t/T)^p \tag{3.15}$$

$$\beta(t) = \beta_0 + (\beta_T - \beta_0) \cdot (t/T)^p \tag{3.16}$$

where t is the current episode, T is the total training duration, and $p = 2$ produces quadratic curves. This schedule ensures strong exploration when the policy is uncertain and gradual transition to stable exploitation as the policy matures.

3.5 AAREN and Probabilistic Belief State

The AAREN module (Attention as Recurrent Neural Network) [19] serves as the core memory and belief state system within the MARQ architecture. It is specifically designed to process extended sequences of partial observations in Stratego, integrating information over hundreds of moves while avoiding the vanishing gradient problem that affects recurrent models like LSTMs or GRUs in long tasks. AAREN leverages an attention mechanism to provide direct, weighted access to the entire history of observations, enabling more coherent and computationally efficient belief tracking [19].

3.5.1 Recurrent Attention Computation

AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Unlike traditional recurrent neural networks that suffer from vanishing gradients over long sequences, AAREN provides direct access to the entire history through attention weights while maintaining O(1) inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.17}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates, while the value v_t encodes the informational content to be integrated into the belief representation. The attention score s_t measures the relevance of the current observation relative to the learned query vector q:

$$s_t = q^T k_t \tag{3.18}$$

The weighted value accumulator a_t aggregates value representations across the sequence:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.19}$$

The term $\exp(m_{t-1} - m_t)$ rescales the previous accumulator when a new maximum is encountered, while $\exp(s_t - m_t)$ computes the relative attention weight for the current observation. The normalization constant $c_t = c_{t-1} \exp(m_{t-1} - m_t) + \exp(s_t - m_t)$ ensures that the effective attention weights sum to unity. The output representation $h_t = a_t / c_t$ represents an attention-weighted aggregation of all observations up to time t.

3.5.2 Piece Type Inference

The AAREN model learns to infer piece values from action sequences using 24-dimensional feature vectors encoding movement patterns, contextual information, and game state features. The network outputs probability distributions over piece types:

$$P(\text{piece\_type} = t \mid \text{action\_sequence}) = \text{softmax}(W_{out} \cdot h_T + b_{out})_t \tag{3.20}$$

This allows the PBS to combine AAREN predictions with rule-based inference and historical pattern matching for robust piece identification under uncertainty.

3.5.3 PBS Quality Evaluation

The MARQ framework incorporates a PBS evaluation system that assesses prediction quality and provides feedback for continuous improvement. The PBS Evaluator employs a neural network architecture that takes belief distributions as input and outputs quality scores $Q_{eval}(\mathcal{B}) = \text{MLP}(\mathcal{B}; \theta_{eval})$. The training objective is based on rewards computed from ground truth comparisons:

$$R_{quality}(\mathcal{B}, g) = \alpha \cdot \mathcal{B}[g] - \beta \cdot |V_{pred} - V_{actual}| - \gamma \cdot \text{distance\_penalty} \tag{3.21}$$

The system determines when additional information gathering is beneficial through uncertainty assessment:

$$\text{NeedMoreInfo}(\mathcal{B}) = ⊮\{H(\mathcal{B}) > \theta_H\} \cdot ⊮\{Q_{eval}(\mathcal{B}) < \theta_Q\} \tag{3.22}$$

3.5.4 Belief Updates on Piece Revelation

When a battle reveals a piece's true type, the PBS synchronizes with ground truth through several update steps. Upon revealing piece type g at position pos, the belief distribution collapses to certainty:

$$\mathcal{B}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.23}$$

The revealed piece count is incremented, and remaining beliefs are adjusted to respect piece count constraints. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max\_count}[t] - \text{revealed\_count}[t]}{\text{remaining\_count}[t]} \tag{3.24}$$

For example, when the single Marshal is revealed, the probability of any other piece being a Marshal drops to zero.

3.5.5 Hindsight-Aided Belief Refinement

While the AAREN module provides inference during gameplay, the Hindsight-Aided Belief Refinement (HABR) module enables learning from verified information. When a piece is revealed through combat, HABR applies retrospective learning across the entire history of that piece, using the ground truth as a supervisory signal for past predictions.

Information Gain Reward. The Information Gain reward quantifies the value of observations that reduce uncertainty about opponent pieces. Let $H(P)$ denote the Shannon entropy of a probability distribution P. The KL divergence from a uniform prior U over N piece types is:

$$D_{KL}(P \parallel U) = \log(N) - H(P) \tag{3.25}$$

The Information Gain reward measures the change in this divergence between consecutive PBS updates:

$$R_{gain}(t) = D_{KL}(PBS_{t-1} \parallel U) - D_{KL}(PBS_t \parallel U) \tag{3.26}$$

Substituting the divergence formula yields:

$$R_{gain}(t) = (\log(N) - H(PBS_{t-1})) - (\log(N) - H(PBS_t)) = H(PBS_{t-1}) - H(PBS_t) \tag{3.27}$$

This result shows that R_{gain} equals the reduction in entropy. A positive reward is achieved only when the observation at time t increases certainty about piece identity compared to $t-1$. This aligns the agent's incentives with active information gathering.

Sinkhorn Constraint Normalization. Independent belief updates for each piece position can lead to inconsistent global predictions. For example, if piece count constraints are not enforced, the model might assign high probability to multiple pieces being the single Marshal. This phenomenon, termed identity inflation, violates the physical constraints of the game.

The Sinkhorn normalization addresses this through iterative proportional fitting. Given a belief matrix $M \in \mathbb{R}^{P \times S}$ where P is the number of tracked pieces and S is the number of piece types, two constraints must hold: the row constraint $\sum_s PBS(p,s) = 1$ ensures each piece has exactly one identity, and the column constraint $\sum_p PBS(p,s) = Count(s)$ ensures the total predicted count of each piece type matches the remaining count.

The Sinkhorn algorithm alternates between row and column normalization until convergence:

$$M^{(k+1)} = D_r^{(k)} M^{(k)} D_c^{(k)} \tag{3.28}$$

$$D_r^{(k)} = \mathrm{diag}\left(\frac{1_P}{M^{(k)} 1_S}\right), \quad D_c^{(k)} = \mathrm{diag}\left(\frac{c}{M^{(k)T} 1_P}\right) \tag{3.29}$$

where c is the vector of remaining piece counts. This operator is differentiable, allowing gradients from the training loss to propagate through the normalization. When piece A is confirmed as the Spy, the Sinkhorn layer automatically forces the Spy probability for all other pieces toward zero, creating a federated update across the entire belief state.

Retrospective KL Divergence. When a piece with identity s_p^* is revealed at time t, the retrospective loss applies across the entire history of that piece:

$$\mathcal{L}_{retro} = \sum_{k=1}^{t} \lambda^{t-k} D_{KL}(PBS_k(p) \parallel \delta_{s_p^*}) \tag{3.30}$$

where $\delta_{s_p^*}$ is the Dirac delta distribution concentrated on the true piece type. The temporal decay factor $\lambda < 1$ (typically 0.9) assigns higher weight to recent predictions.

The decay factor addresses the non-stationary nature of piece behavior in Stratego. A piece may exhibit deceptive behavior in the early game, such as a Marshal moving conservatively to avoid revealing its identity, before adopting more characteristic patterns in the endgame. By weighting recent observations more heavily, the loss function prioritizes learning from high-information behaviors near the revelation event while still providing a smoothed learning signal over the piece's history.

3.6 Multi-Agent Learning and Strategy Mixing

MARQ employs a league reinforcement learning system that minimizes non-stationarity, a fundamental challenge in multi-agent learning where policy updates by one agent alter the environment experienced by others.

3.6.1 League-Based Training Architecture

The league consists of a dynamic pool of agents designed to ensure robust generalizability [44]. The main agent represents the current best-performing policy that is being actively optimized through gradient descent. The historical pool contains saved collections of past main agent checkpoints representing distinct strategic eras discovered during training.

During training, opponents are sampled using a probability distribution:

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.31}$$

This prevents cycling where the agent re-learns and un-learns the same strategies endlessly, forcing it to defeat all previous versions of itself.

3.6.2 Population-Based Training

Population-Based Training (PBT) extends the league training paradigm by treating hyperparameter optimization as a parallel evolutionary process. Rather than training a single agent with fixed hyperparameters, PBT maintains a population of K workers, each training independently with potentially different hyperparameter configurations.

At regular intervals (typically hourly), the supervisor evaluates the population and performs exploitation: workers in the bottom quantile are terminated, and their training is restarted using weights cloned from top-performing workers. This mechanism allows the population to collectively discover effective hyperparameter schedules during training rather than requiring extensive tuning prior to training.

To prevent the population from collapsing to a single strategy, cloned workers receive perturbed hyperparameters:

$$\theta_i' = \theta_i \cdot \xi, \quad \xi \sim U(0.8, 1.2) \tag{3.32}$$

The exploration rate ϵ is reset to a non-zero value (typically 0.1) for cloned workers, enabling renewed exploration from the inherited policy. This combination of exploitation (copying successful weights) and exploration (perturbing hyperparameters) allows PBT to adapt the training configuration as the policy evolves, addressing the observation that optimal hyperparameters change over the course of training.

3.6.3 Strategy Mixing for Exploitation Resistance

Extensive-form imperfect information games with deterministic strategies are inherently suboptimal and vulnerable to exploitation [36, 46]. An adversary with knowledge of a deterministic policy effectively reduces the game to a perfect information setting, nullifying the strategic uncertainty fundamental to games like Stratego. The MARQ framework addresses this challenge through two complementary mechanisms.

At the action selection level, MARQ employs Noisy Linear layers that inject learned, state-dependent noise into the policy. Rather than using fixed exploration schedules such as ϵ-greedy, the network learns the appropriate degree of randomization for each game state.

At the strategy level, MARQ achieves mixing through diverse opponent exposure during training. The league system maintains a population of historical agent checkpoints representing different strategic approaches discovered during training. The combination of action-level noise and strategy-level diversity produces policies that are both unpredictable in individual games and robust across different opponent types.

3.6.4 Multi-Dimensional Reward Structure

The reward function $R(s, a, s')$ is shaped to guide learning in the sparse-reward Stratego environment [17]:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \quad (3.33)$$

The move reward R_{move} provides small rewards for advancing pieces (approximately +0.05) to encourage active play. The capture reward $R_{capture}$ is scaled by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$. The tactical reward $R_{tactical}$ gives specific bonuses for key strategic captures, including +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces.

3.7 Specialized Agent Architectures

3.7.1 Setup Agent via Distributional RL

The game's initial setup is handled by a specialized SetupAgent that learns optimal piece configurations by treating the placement phase as a sequential decision process. The state representation consists of a 3-channel input tensor encoding the current board state with already-placed pieces, a valid positions mask indicating legal placement locations, and a current piece type mask identifying which piece is being placed.

The agent outputs a distributional Q-value for each of the 100 board positions, mapping $Q(s, pos)$ to a distribution over $[V_{min}, V_{max}]$. This distributional approach captures the inherent uncertainty in setup quality, where the true value of a placement depends on the unknown opponent configuration.

To prevent creating static, easily countered setups, the agent includes an exploitability critic network that penalizes predictable placements:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \lambda \cdot \mathcal{L}_{predictability} \quad (3.34)$$

3.7.2 Information Set Monte Carlo Tree Search (IS-MCTS)

For high-stakes decision making, particularly in the endgame, MARQ employs an Information Set MCTS agent [13] that combines search with learned value functions. The search process proceeds in three phases.

First, during the determinization phase, the agent samples a consistent concrete world state s_{det} from the PBS beliefs according to $s_{det} \sim P(S|\mathcal{B}_t)$. This converts the imperfect information game into a perfect information instance for the duration of the simulation.

Second, during tree traversal, the search tree is navigated using a UCB1 variant to balance exploration and exploitation of move sequences. Nodes in the tree represent information sets rather than individual states, allowing the tree to be shared across multiple determinizations.

Third, during leaf evaluation, terminal or unexpanded nodes are evaluated using the Rainbow DQN value function rather than random rollouts:

$$V(s_{leaf}) \approx Q_{DRQN}(s_{leaf}, a_{best}) \tag{3.35}$$

3.8 Game Environment and State Management

The Stratego environment provides the foundation for agent training and evaluation, managing the complex dynamics of imperfect information.

3.8.1 Information Set Management

The environment maintains distinct representations for different information contexts:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.36}$$

Each player receives only the subset of game state information available through their observation function.

3.8.2 Temporal State Evolution

Game states evolve according to the update function:

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.37}$$

where θ_t represents stochastic elements (hidden information revelation, time limits, etc.).

3.8.3 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.38}$$

This ensures that agents can only select legal moves consistent with Stratego's rules and piece capabilities.

3.8.4 Piece Setup and Initialization

The initial game setup involves strategic piece placement governed by setup policies:

$$\pi_{setup}(\text{placement}|player) = \text{NeuralNetworkSetup}(\text{strategic\_features}, \theta_{setup}) \tag{3.39}$$

where strategic features include positional value maps, piece interaction potentials, and defensive vulnerabilities. The setup phase represents a critical strategic decision that influences the entire game trajectory [15].

3.9 Convergence Properties

The MARQ framework provides theoretical guarantees for convergence and performance under specific conditions.

3.9.1 Incremental Learning Bounds

Under standard RL assumptions (independent identical distribution of experiences, bounded rewards), the MARQ algorithm provides the following bound on value function estimation:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.40}$$

where C depends on reward bounds and discount factor, and ϵ_{approx} represents function approximation error.

3.9.2 Belief State Accuracy Bounds

The PBS accuracy improves with additional observations according to:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.41}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

3.9.3 Strategy Convergence in Self-Play

The league training methodology ensures convergence to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T\to\infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.42}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity.

The league training system ensures improvement through three complementary mechanisms. First, the system maintains best response values against historical opponents, ensuring that the current policy remains competitive against the full distribution of strategies encountered during training. Second, promotion to the main agent pool requires a positive win rate against previous versions before replacement, preventing regression in overall performance. Third, diversity bonuses in the opponent selection distribution prevent premature convergence to strategies that may be exploitable by counter-strategies.

These theoretical foundations provide confidence in the MARQ framework's ability to learn effective strategies in the challenging domain of imperfect information strategic gameplay while remaining computationally tractable.

Chapter 4

Methodology

This chapter outlines the methodology employed to evaluate the effectiveness of the MARQ framework in solving mind sports with imperfect information. The approach involves the development of a competitive system, followed by the evaluation of its performance through win-rate analysis against human players and an assessment of its adaptability across diverse game environments. The subsequent sections detail the processes and methods that will be used to achieve these research objectives.

4.1 Project Planning

This research will be conducted according to a structured plan encompassing participant selection, sample size determination, and experimental procedures. The project will follow a rapid prototyping approach for the design and implementation of the MARQ framework. The timeline is segmented into four distinct phases. The development phase will run from September 2025 to February 2026, beginning with game environment development from September through November 2025, followed by agent training from December 2025 to January 2026, and concluding with system deployment in February 2026. The user testing phase will take place in March 2026, involving one-on-one matches and data collection activities. The final analysis and documentation phase will commence in April 2026, focusing on data analysis and manuscript preparation.

4.1.1 Community Introduction to Stratego

The introduction of the game Stratego to the participant community is a prerequisite for this study. This necessity arises from a complete lack of available local participants possessing prior knowledge of the game's rules or strategies. Unlike more popular games like Chess, Stratego's niche status means a pre-existing pool of experienced players is not available for participation to test against the MARQ framework. Therefore, a structured introduction is essential to cultivate the necessary participant cohort from the ground up. This approach ensures all participants begin with a uniform baseline understanding. It allows for the parallel development of both human expertise and artificial intelligence, creating a synchronized learning environment where the adaptive capabilities of the MARQ framework can be fairly assessed against a human cohort that is also in the process of learning.

4.1.2 Participant Selection

The participant selection process is designed to recruit a cohort of 20 individuals via convenience sampling, taking into account the project's resource constraints while still yielding a sufficient sample. The target demographic is enthusiasts of strategic mind sports of a diverse age and game experience.

Prospective participants will be screened to evaluate two key attributes. Their strategic game experience is key to ensuring a baseline aptitude, and second, their confirmed lack of prior knowledge of Stratego to ensure the study's "synchronized learning" premise. Ultimately, only those who successfully finish the screening phase will be admitted, guaranteeing they possess the necessary foundational skills to provide meaningful interaction with the MARQ framework and substantive feedback.

4.1.3 Experimental Procedures

One-on-one matches between human participants and the MARQ framework will be conducted on-site using a single device. To ensure consistency, each participant will receive a checklist outlining the procedures for interacting with the game.

Each session will last one (1) hour, during which participants must complete at least two (2) games against the MARQ framework. Additional games may be played if the time permits. The following game statistics will be recorded will include the winner, total game duration, total moves per player, and the number and type of remaining pieces for both players. Immediately following the session, each participant will be given an evaluation form to provide open-ended feedback on their experience.

4.2 System Development

The system development consists of two components: the game environment and the MARQ framework. The game environment will be developed to serve as the interface for human players and the world in which the trained agent operates. The MARQ framework provides the trained agent that competes against human players.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, providing a deterministic alternative to empirical estimation through gameplay simulation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined as:

$$\mathrm{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \mathbb{1}_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\mathbb{1}_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with exceptions: the Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is:

$$\mathrm{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \nVdash_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

where $\nVdash_{p \succeq a}$ equals 1 if piece p survives an attack from piece a (i.e., p defeats a or both are destroyed).

Auxiliary Value Components

Three additional factors contribute to piece value:

Strategic Ability Bonus. Certain pieces possess capabilities that extend beyond their combat rank. Table 4.1 enumerates these bonuses.

Table 4.1: Strategic ability bonuses for pieces with special capabilities.

Piece	**Bonus**	**Description**
Miner	2.0	Sole piece capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

Mobility Factor. The Scout receives a mobility bonus of 1.0 due to its ability to move multiple squares per turn; all other movable pieces receive a base mobility of 0.5. Immovable pieces (Flag, Bomb) receive 0.

Scarcity Factor. The scarcity of each piece type influences its strategic importance. This is computed using logarithmic scaling:

$$\mathrm{SF}(p) = \log\left(\frac{\max_{i \in \mathcal{P}} n_i}{n_p}\right) + 1 \tag{4.3}$$

Composite Value Function

The final piece value is a weighted linear combination of the above components:

$$V(p) = \alpha_1 \cdot \mathrm{CDS}(p) + \alpha_2 \cdot \mathrm{SR}(p) + \alpha_3 \cdot S(p) + \alpha_4 \cdot M(p) + \alpha_5 \cdot \mathrm{SF}(p) \tag{4.4}$$

where $S(p)$ is the strategic ability bonus and $M(p)$ is the mobility factor. The weights $\alpha = (0.40, 0.25, 0.20, 0.10, 0.05)$ are selected to prioritize combat capability while accounting for strategic factors. Values are normalized such that $V(\mathrm{Scout}) = 1.0$, analogous to the convention in chess where the Pawn serves as the unit of measurement.

Table 4.2: Computed piece values with associated combat and survival metrics.

Piece	**Value**	**CDS**	**SR**
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Bomb†	2.0	—	—
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030
Flag‡	0.1	—	—

†Immovable defensive piece; eliminates any non-Miner attacker.

‡Immovable objective piece; capture results in game loss.

These values inform the material reward component of the reward shaping function described in subsequent sections.

4.2.2 Handling Imperfect Information: Probabilistic Belief State (PBS)

A fundamental challenge in Stratego is the hidden identity of opponent pieces. To address this, the system incorporates a Probabilistic Belief State (PBS), represented as a matrix that maintains a probability distribution over the possible identities of each unrevealed opponent piece. The PBS is updated throughout the game through two complementary reasoning mechanisms.

Deductive reasoning applies when direct piece interactions occur. Upon combat, the identity of the involved pieces becomes known with certainty, and the PBS is updated accordingly. The probability mass previously assigned to eliminated possibilities is redistributed among remaining candidates through renormalization.

Inductive reasoning enables the system to infer piece identity from observed behavior patterns. Pieces exhibiting high mobility and long-range movement are assigned increased probability of being Scouts. Conversely, pieces that retreat from threats or remain stationary near the back rank suggest defensive behavior characteristic of high-value pieces such as the Flag or Marshal. These behavioral heuristics are incorporated through Bayesian updates to the belief distribution.

Hindsight-Aided Belief Refinement. Training the PBS module presents a challenge: ground truth labels are only available when pieces are revealed through combat, which may occur many moves after the initial observations. The Hindsight-Aided Belief Refinement (HABR) module addresses this by storing PBS snapshots throughout each piece's history. When a piece is revealed with identity s_p^*, HABR computes a retrospective loss:

$$\mathcal{L}_{retro} = \sum_{k=1}^{t} \lambda^{t-k} D_{KL}(PBS_k(p) \parallel \delta_{s_p^*}) \tag{4.5}$$

where $\lambda = 0.9$ is the temporal decay factor. This loss propagates the verified signal backward through the prediction history, training the belief tracker to recognize patterns that predict the revealed identity. The decay factor prioritizes recent observations while maintaining gradient flow to earlier predictions.

Sinkhorn Global Constraints. To prevent identity inflation where multiple pieces are assigned high probability for the same rare type, a Sinkhorn normalization layer enforces global piece-count constraints. After each PBS update, the Sinkhorn operator iteratively adjusts the belief matrix such that: each piece's probabilities sum to 1, and the total predicted count for each piece type matches the remaining count. This differentiable normalization enables end-to-end training while maintaining physically consistent predictions.

Information Gain Reward. The PBS module computes an Information Gain reward that measures entropy reduction between consecutive updates: $R_{gain}(t) = H(PBS_{t-1}) - H(PBS_t)$. This reward incentivizes the agent to make moves that gather information about opponent pieces, balancing exploitation with active deduction.

4.3 Model Training

Training the MARQ agent is computationally intensive due to the large state-action space and the need to learn accurate belief state representations over extended game histories.

Rainbow DQN Architecture. MARQ employs a Rainbow DQN architecture that integrates several algorithmic improvements over the standard DQN. The implementation includes Categorical DQN (C51) for distributional value estimation with 51 atoms over the support $[V_{min}, V_{max}] =$

$[-15, 15]$, Noisy Networks for learned exploration (replacing ϵ-greedy), and Dueling Network Architecture separating state value and action advantage estimation. The noisy linear layers inject state-dependent stochasticity, enabling the agent to learn appropriate exploration levels for different game situations without manual scheduling.

Network Structure. The Q-network employs a Deep Residual Network (ResNet) backbone to capture long-range spatial dependencies across the board. The architecture consists of an initial convolutional layer (64 filters, 3×3 kernel) followed by six residual blocks (twelve convolutional layers total), each containing two convolutional layers with Instance Normalization and ReLU activation. This deep feature extractor feeds into the dueling streams, allowing for more abstract strategic reasoning compared to shallow architectures. The 27-channel state representation is partitioned into two components, as detailed in Table 4.3.

Table 4.3: State representation input channels for the Rainbow DQN.

Channel	**Category**	**Description**
Board Features (Channels 0–14)		
0–11	Own Pieces	Binary spatial maps indicating positions of each piece type (Flag, Spy, Scout, Miner, Sergeant, Lieutenant, Captain, Major, Colonel, General, Marshal, Bomb)
12	Enemy Mask	Binary map indicating positions of all visible enemy pieces
13	Obstacles	Binary map of lake squares (impassable terrain)
14	Empty Squares	Binary map of unoccupied traversable positions
Enemy Type Features (Channels 15–26)		
15–26	Piece Type Probabilities	During full observability phases, these channels contain one-hot ground truth encodings of enemy piece types; during partial observability phases, they contain PBS probability distributions indicating the inferred likelihood of each piece type at enemy positions

The board feature channels provide deterministic knowledge of the agent's own piece positions and observed enemy locations. The enemy type channels encode information about hidden enemy piece identities, with the representation varying by training phase. During full observability training (Phase 1), these channels contain ground truth one-hot encodings that allow the agent to learn fundamental game mechanics and piece-type awareness without inference uncertainty. During partial observability phases, the channels contain probabilistic beliefs from the PBS module, updated through the AAREN attention mechanism based on observed opponent behavior. This dual-mode representation enables the network to first acquire robust combat and positional strategies before learning to operate under uncertainty. The AAREN module uses 24-dimensional action feature vectors, 64 hidden units, and 3 layers to process action histories and output piece type probability distributions.

Training Hyperparameters. The training configuration uses several key parameters: learning rate $\alpha = 3 \times 10^{-5}$ with AdamW optimizer and weight decay $\lambda = 0.01$, and a discount factor $\gamma = 0.995$ to capture long-term strategic consequences. We utilize a batch size of 512 transitions and a replay buffer capacity of 150,000 transitions with prioritized sampling. Soft target network updates ($\tau = 0.001$) occur every 5,000 steps. Model updates are performed every 2 steps utilizing 8 parallel lanes and 3-step returns to improve credit assignment. To improve sample efficiency, data augmentation is applied via horizontal flip, exploiting the game's symmetry. Training proceeds for 35,000+ episodes with checkpoints saved every 1,000 episodes.

Computational Optimizations. To accelerate training throughput, the implementation employs mixed-precision inference using PyTorch's automatic mixed-precision (AMP) framework. During action selection, network forward passes operate in FP16 precision via `torch.amp.autocast`, reducing memory bandwidth requirements and leveraging Tensor Core acceleration on NVIDIA GPUs. The replay buffer stores transitions in FP16 format, reducing GPU memory consumption by approximately 50% compared to FP32 storage. Additionally, CUDNN benchmark mode is enabled to automatically select the fastest convolution algorithms for the fixed input tensor dimensions.

League-Based Training. To prevent policy cycling, the agent trains against a diverse opponent pool. The opponent selection distribution allocates 50% to historical league agents, 20% to random agents, 20% to greedy heuristic agents, and 10% to self-play. Agent checkpoints are added to the league every 500 episodes, with a maximum pool size of 50 historical agents.

Population-Based Training. To accelerate hyperparameter optimization during training, the framework implements Population-Based Training (PBT). PBT combines the advantages of parallel random search with online model selection by maintaining a population of 4 to 8 independent training workers, each initialized with a unique random seed to ensure exploration diversity.

A supervisor process monitors worker performance by reading metrics from a shared log file every 30 seconds. Every hour, the supervisor evaluates all workers based on their mean reward over the most recent 100 episodes and performs exploitation: the bottom 50% of workers are terminated, and new workers are launched using cloned weights from the top performers. To encourage continued exploration, cloned workers receive perturbed hyperparameters according to:

$$\theta_i' = \theta_i \cdot U(0.8, 1.2) \tag{4.6}$$

where $U(0.8, 1.2)$ denotes a uniform random perturbation factor. The exploration rate ϵ is reset to 0.1 for cloned workers, allowing renewed discovery from the inherited policy. This adaptive schedule enables the population to collectively discover effective hyperparameter configurations during training rather than requiring extensive manual tuning prior to training.

Curriculum Learning. Training follows a five-phase curriculum of increasing complexity with extended episode counts to ensure thorough learning at each stage. Phase 1 (Full Observability) allows the agent to observe all piece identities, enabling acquisition of fundamental game mechanics. To mitigate the "observability cliff" when transitioning to Phase 2, a Mixed Observability mechanism is introduced: as the agent's win rate exceeds 85% in Phase 1, a stochastic "Fog of War" randomly occludes 50% of opponent pieces. This forces the agent to begin relying on probabilistic inference and PBS predictions before full occlusion occurs.

Training proceeds for 5,000–10,000 episodes against a progressively challenging opponent mix: initially random opponents, then basic heuristic agents, and finally a sophisticated multi-factor heuristic opponent. Phase advancement requires ≥95% win rate against random opponents, ≥75% against the basic heuristic, and ≥60% against the strategic heuristic.

Phase 2 (Partial Observability) disables full observability, requiring reliance on PBS predictions for 5,000–10,000 episodes. The opponent distribution allocates 40% to frozen heuristic agents and 60% to the strategic heuristic opponent. Phase advancement requires PBS prediction accuracy ≥75% and overall win rate ≥60%.

Phase 3 (Self-Play) trains against a mixture of delayed agent copies (60%) and strategic heuristic opponents (40%) for 5,000–15,000 episodes to discover novel strategies while preventing policy collapse. Phase 4 (League Training) activates the full opponent distribution with 40% historical league agents, 30% strategic heuristic, 20% greedy agents, and 10% self-play. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

4.3.1 Centralized Reward System and Defensive Normalization

The reward function is the primary driver of agent behavior, transforming tactical events into learning signals. To ensure consistency and numerical stability, the framework employs a centralized `UnifiedRewardShaper` that consolidates all reward logic.

Composite Reward Mixing. The framework aggregates four distinct reward categories using a weighted linear combination:

$$R_{total} = w_o R_{outcome} + w_m R_{material} + w_e R_{epistemic} + w_p R_{positional} \tag{4.7}$$

where weights are established as $w_o = 1.0$, $w_m = 0.5$, $w_e = 0.1$, and $w_p = 0.05$. This hierarchy ensures that the primary objective (winning) remains paramount, while auxiliary tactical signals are strictly limited to guidance and cannot accumulate to magnitudes that incentivise reward hacking.

Rationale for Distributional Scaling. The choice of support range $[V_{min}, V_{max}] = [-15, 15]$ is calibrated to accommodate boosted terminal rewards. With 51 atoms, the distribution resolution is $\delta_z = 0.6$, providing sufficient granularity while allowing the full range of expected returns to be represented. Terminal outcomes are configured such that win and loss rewards are set to ± 10.0 to provide a strong learning signal in this sparse reward environment, while a draw receives -1.0, indicating it is preferable to a loss but inferior to a win. A constant step penalty of -0.00005 per step provides light time pressure; over a 4000-step game, this accumulates to -0.2, which does not overwhelm the terminal reward. For positional constraints, positional rewards for territory advancement are state-based to prevent movement farming, with the agent receiving a one-time bonus of $+0.05$ when reaching a new row closer to the enemy base. Dense rewards for information gain (revealing enemy pieces) and center control provide guidance when the terminal reward is distant, serving as sparse reward mitigation.

Architectural Evolution and Implementation History. The implementation of this reward system followed an iterative trajectory to resolve scaling anomalies and logic redundancy. Early legacy versions performed reward calculations directly within the game environment (`environment.py`), supplemented by independent shaping modules (`reward_shaping.py` and `aggression_reward.py`). This led to logic fragmentation, where hidden step penalties and overlapping reward signals interfered with the training process, occasionally resulting in large reward spikes that overwhelmed the

C51 distribution. The current consolidated architecture utilizes a unified `UnifiedRewardShaper` module. By consolidating disparate shaping logic into a single point of truth, the framework ensures that all rewards are properly normalized and mutually consistent. The environment now acts as an objective state manager, returning only raw event data in an *info* dictionary, which the shaper transforms into a stable learning signal aligned with the technical requirements of the Rainbow DQN.

Positional and Anti-Stall Mechanisms. Beyond combat, the system implements specific shaping to guide behavior. A constant base step penalty of -0.0001 creates a slight time pressure toward efficiency. For stalemate prevention, sudden mobility restrictions (dropping below 5 moves) trigger a -0.05 penalty, discouraging the agent from boxing itself into immovable positions. Strategic proximity shaping grants a small $+0.05$ bonus for moving pieces toward the opponent's back rank (especially corners), guiding the agent toward typical flag locations.

Experience Replay Optimization. Standard uniform sampling from the replay buffer treats all transitions equally, which is sample-inefficient when important learning events (e.g., flag captures, high-value combat) occur infrequently. Prioritized Experience Replay (PER) addresses this by sampling transitions with probability proportional to their temporal-difference (TD) error magnitude:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon \tag{4.8}$$

where δ_i is the TD-error for transition i, $\alpha \in [0, 1]$ controls the degree of prioritization (with $\alpha = 0.6$ used in this implementation), and $\epsilon = 10^{-6}$ prevents zero probability. To correct the bias introduced by non-uniform sampling, importance sampling weights are applied:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta \tag{4.9}$$

where β is annealed from 0.4 to 1.0 over training to gradually enforce unbiased updates. The implementation uses a sum-tree data structure for $O(\log N)$ sampling complexity. PER increases sample efficiency by focusing learning on transitions where the current value function is least accurate.

Advantage Filtering. In standard deep reinforcement learning, the experience replay buffer stores past transitions (state, action, reward, next state) and samples them uniformly during training. However, not all experiences are equally valuable for learning. A routine move that shuffles a piece sideways provides less learning signal than a decisive capture that changes the game trajectory. Building upon Prioritized Experience Replay, the framework implements advantage filtering inspired by the Ataraxos system for imperfect-information games.

The key insight is that transitions where the value network makes larger prediction errors indicate situations the agent does not yet understand well. These high-error transitions are precisely where gradient updates will produce the most learning improvement. The filtering procedure works as follows: first, the replay buffer is oversampled by a factor of 4, meaning if the target batch size is 128 transitions, the system initially samples 512 candidates. Each candidate's temporal-difference (TD) error is computed, which measures how much the predicted value differs from the observed outcome. The 512 candidates are then ranked by the absolute magnitude of this error, and only the top quartile (128 transitions with the largest errors) is retained for the actual gradient computation. This selective training focuses learning on the most informative experiences while maintaining the same computational cost per gradient step.

Dynamic Damping Regularization. A common problem in multi-agent reinforcement learning is strategy cycling, where the policy oscillates between different approaches without converging to a stable solution. For example, an agent might learn an aggressive attacking strategy, then shift to a defensive strategy when it encounters difficulties, then return to attacking when the defensive approach fails. This oscillation wastes training time and prevents the discovery of robust strategies.

To address this problem, the loss function incorporates two additional regularization terms based on Kullback-Leibler (KL) divergence. KL divergence measures how different two probability distributions are from each other. The first regularization term, called the magnet term, measures the difference between the agent's current action probabilities and a uniform distribution where all actions are equally likely. This term encourages exploration by penalizing policies that become too confident in specific actions. The second term, called the target anchor, measures the difference between the current policy and the more stable target network policy. This term provides stability by preventing the policy from changing too rapidly.

The key innovation is that these two terms have opposing schedules that change over training. The magnet coefficient starts at $\alpha_0 = 0.1$ and decays to 0.001 following a quadratic curve, meaning exploration is strongly encouraged early in training when the agent knows little about the game. The target anchor coefficient starts at $\beta_0 = 0.001$ and grows to 0.1, meaning stability becomes increasingly important as the agent develops a competent policy. This schedule ensures that the agent explores aggressively during initial learning phases and gradually transitions to stable exploitation as training progresses.

Multi-Step Returns. The standard one-step TD target $r_t + \gamma V(s_{t+1})$ can exhibit high variance and slow credit assignment for sparse rewards. Multi-step returns extend the bootstrapping horizon to n steps:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a^*) \tag{4.10}$$

where $a^* = \arg\max_a Q(s_{t+n}, a)$ under Double Q-learning. With $n = 5$ and $\gamma = 0.995$, the effective discount for the bootstrapped value is $\gamma^5 \approx 0.975$. This configuration drastically reduces the time constant for reward propagation, enabling the terminal win/loss signals to influence early-game tactical decisions more effectively than shallow horizons.

Learning Rate Scheduling. Convergence can be accelerated through adaptive learning rate control. A step decay schedule reduces the learning rate by factor $\eta = 0.9$ every 5,000 episodes:

$$\alpha_e = \alpha_0 \cdot \eta^{\lfloor e/5000 \rfloor} \tag{4.11}$$

where $\alpha_0 = 10^{-4}$ is the initial learning rate and e is the episode count. This schedule enables aggressive early learning while maintaining stability as the policy approaches convergence.

4.3.2 Action Space Reduction via Heuristic Filtering

The full action space in Stratego consists of 400 discrete actions, encoding each starting position on the 10×10 board paired with four cardinal directions. However, many of these actions are either illegal (moving off-board or into lakes) or strategically irrelevant at any given game state. Presenting all 400 outputs to the network, even with invalid action masking, introduces noise into the learning process as the network must learn to distinguish between many low-value alternatives.

Heuristic Move Scoring. To address this, a heuristic pre-filter scores all legal moves and selects the top-k candidates (where $k = 100$ by default) for consideration. The scoring function assigns points based on the following criteria:

Table 4.4: Heuristic scoring criteria for move filtering.

Criterion	**Score**	**Rationale**
Attack (target is enemy)	+100	Captures are always strategically relevant
Forward advancement	+5/row	Territorial progress toward enemy base
Center control (cols 3–6)	+10	Central positioning provides flexibility
Retreat	−20	Backward movement is generally defensive
Scout distance (> 1 square)	+2/sq	Long-range reconnaissance value

Action Masking Integration. Rather than modifying the network architecture, the filtering is implemented through action masking at inference time. The network retains its 400-dimensional output, preserving consistent action semantics throughout training. For each decision:

1. All legal moves are scored using the heuristic function.
2. Moves above a threshold (score > 50) are unconditionally included.
3. The remaining positions are filled from the next highest-scoring moves until 100 actions are selected.
4. A binary mask is constructed where selected actions receive weight 0 and excluded actions receive $-\infty$.
5. The mask is added to the Q-value output before argmax selection.

This approach ensures that attack moves and forward advances are always available for selection, while low-value shuffling moves are pruned. The fixed output dimensionality prevents the representational instability that would occur if the network had to learn different action encodings across game states.

4.3.3 Deployment

Upon completion of the training and validation cycles, the best-performing MARQ model weights will be saved and finalized for deployment. Deploy the trained model into the Stratego game environment described in Section 2 of the methodology.

The deployed system will function as the complete application used during the user testing phase. It will be configured to run locally on the single device designated for the experimental procedures. In this production mode, the model's exploratory functions will be deactivated for a more measured

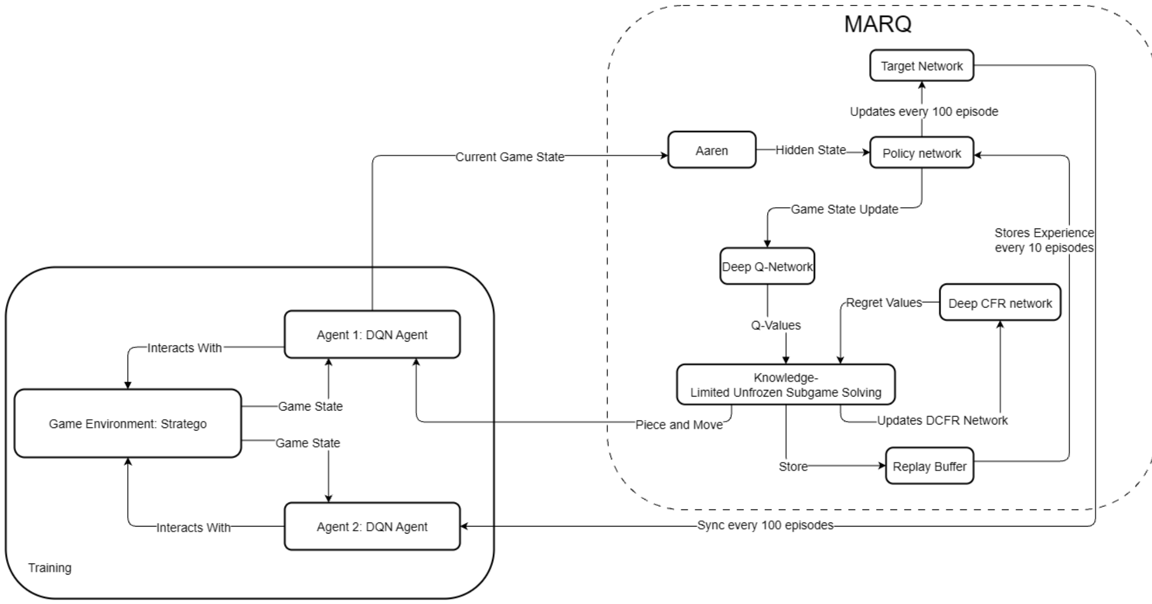

Figure 4.1: The MARQ System Architecture, which takes the Board State and PBS as input.

approach from the agent, always selecting the action with the highest predicted Q-value. This ensures that all human participants compete against the identical, optimized version of the MARQ framework, providing a consistent baseline for the quantitative and qualitative evaluations.

In the figure above, the player starts the game and the loop begins until either the player wins by capturing the opponent's flag or loses if the agent captures the player's flag. The agent receives the current board state and updates the AAREN module, which constructs the PBS representing the probability distribution over hidden piece identities. The DQN then evaluates Q-values for all legal actions given this belief-enhanced state representation and selects the action with the highest expected value.

4.4 Evaluation

A comprehensive evaluation approach combining quantitative and qualitative methods will be used to assess the MARQ framework's performance and the user experience.

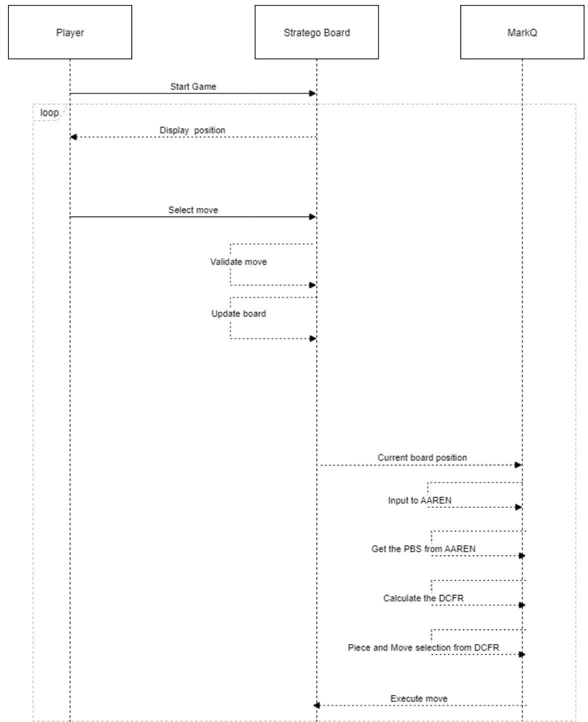

Figure 4.2: Game Sequence Diagram

Quantitative Analysis. The quantitative assessment of the model's performance tracks several key metrics to gauge learning progress and strategic capability. Win rate percentage measures the proportion of games won against human participants. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Decision-making speed records the average time taken by the agent to select an action. Training stability is assessed through loss curves showing the progression of distributional and value losses over training episodes. To evaluate the synchronized learning environment, a two-tailed paired t-test will determine if there is a statistically significant difference in gameplay metrics between a player's first and last games, assessing whether human players improve against the AI. The results from Likert-scale questions will be analyzed using a weighted mean to derive a quantitative score for each dimension of user acceptance.

Qualitative Analysis. To assess end-user opinion on usability, a usability testing session will gather participant feedback through an evaluation form evaluating three key dimensions. Performance assessment examines the agent's speed, consistency, and perceived strategic competence.

Usability evaluation considers the ease of use, intuitiveness of the interface, and overall user experience. Relevance gauges the AI's value as a strategic training tool and its potential for helping players learn Stratego.

Model Iteration and Initial Results. The development of the MARQ framework is an iterative process where each model version represents a snapshot of the system's evolution. Appendix B presents initial training results from 1,000 episodes. During this early phase, the agent remains in an exploratory state, searching for exploitable strategies. The reward signal indicates difficulty in discovering effective tactics against the opponent. Early win rates reflect frequent draws caused by repetitive piece movements. With the transition to Noisy Networks, the agent's stochasticity is learned rather than scheduled, allowing more adaptive exploration throughout training.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

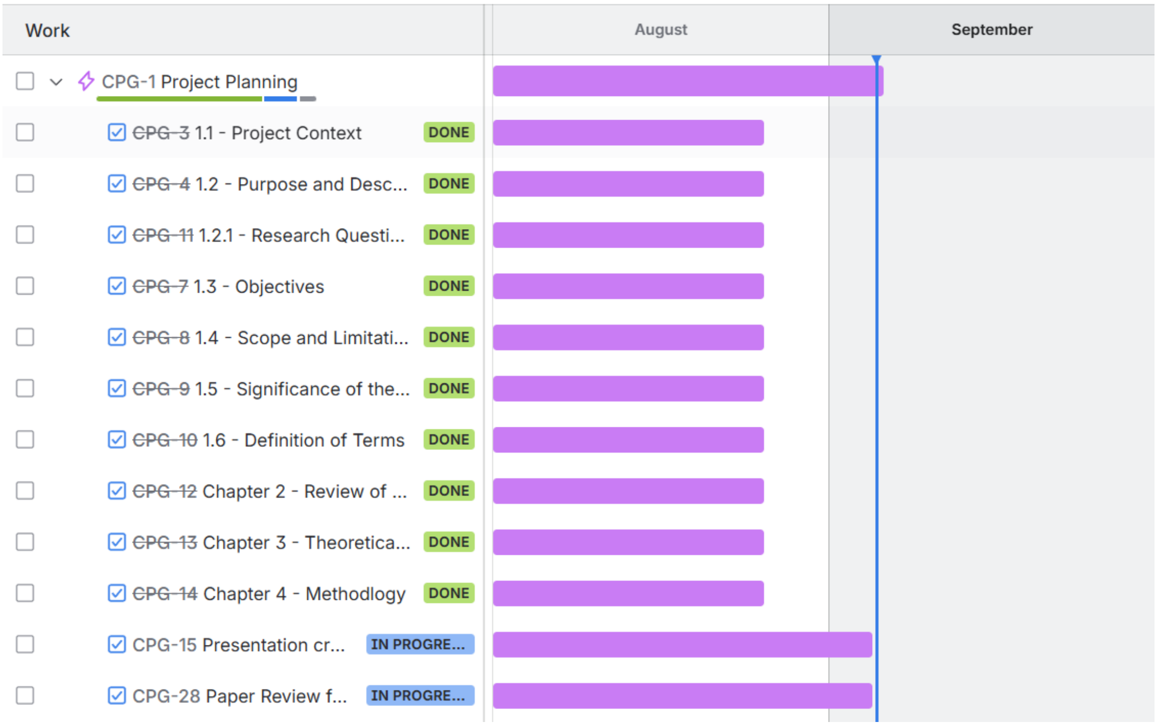

Figure A.1: Timeline: August 1 - September 5

Figure A.2: Timeline: September 11 - November 30

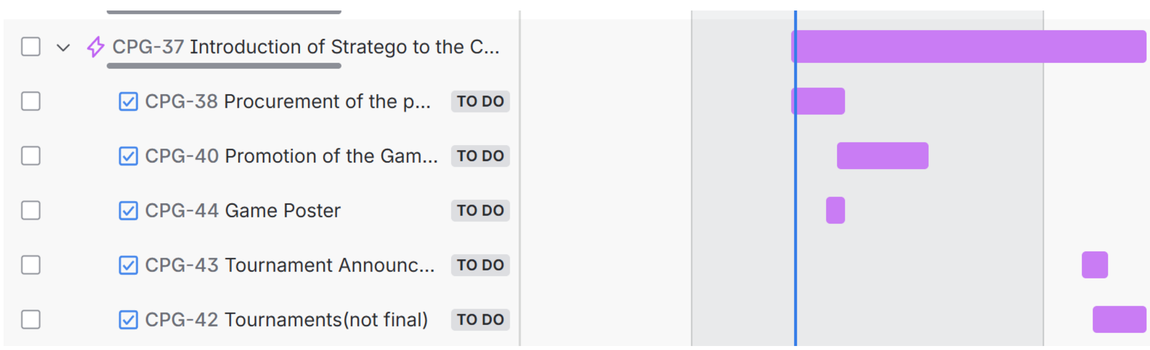

Figure A.3: Timeline: October 27 - January 27

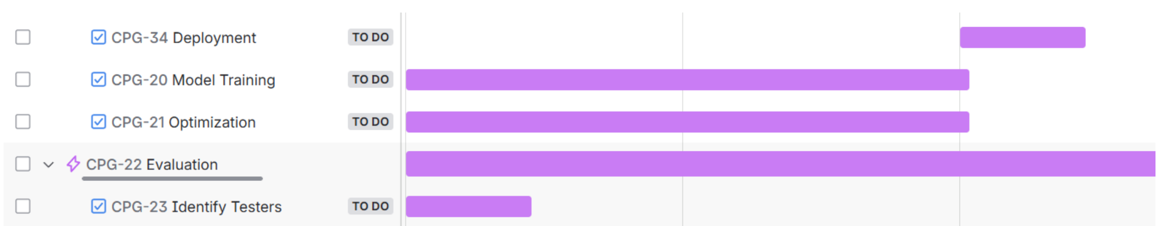

Figure A.4: Timeline: December 1 - February 14

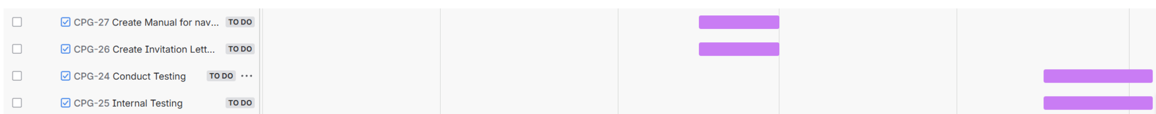

Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

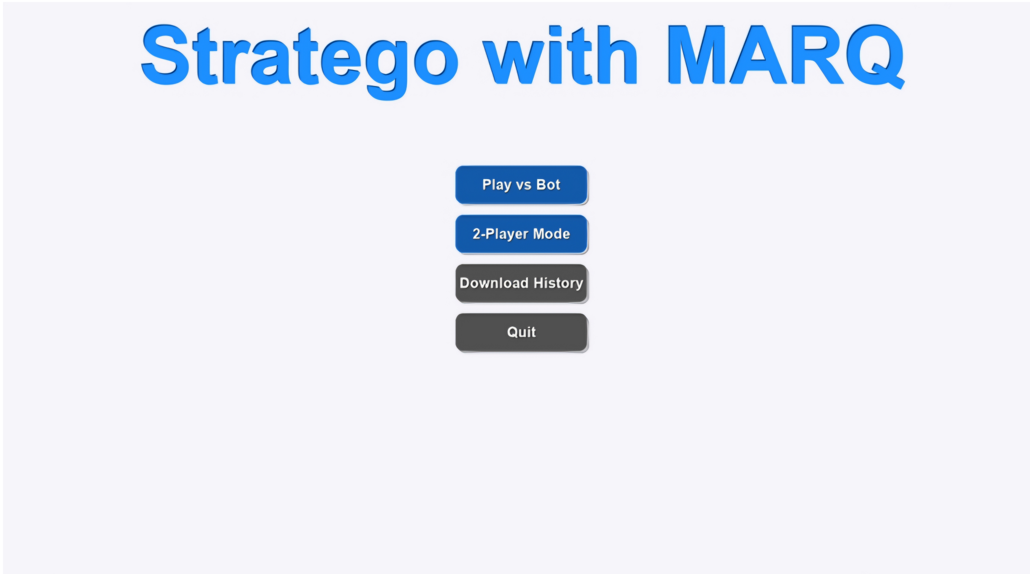

Figure B.1: Game Menu

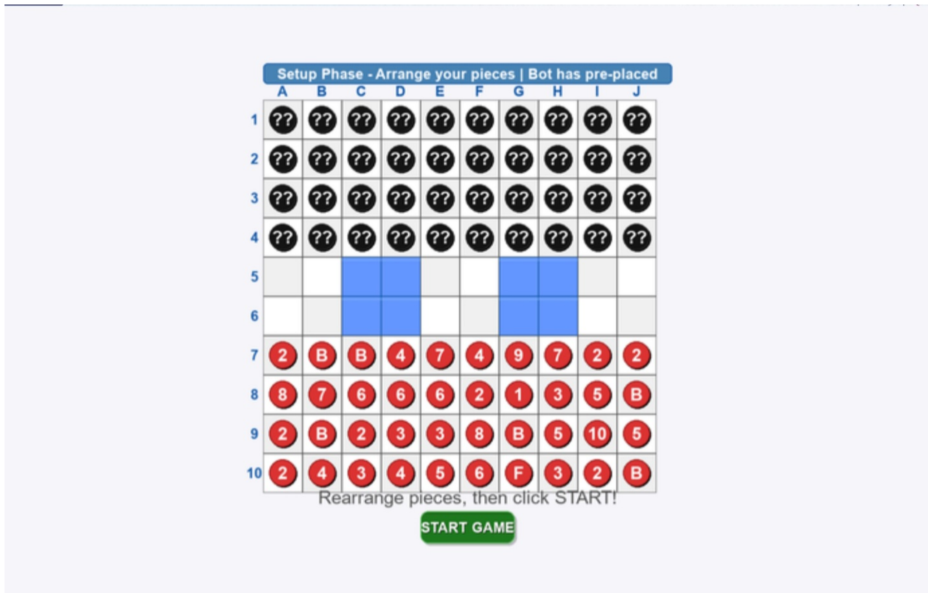

Figure B.2: Setup Phase

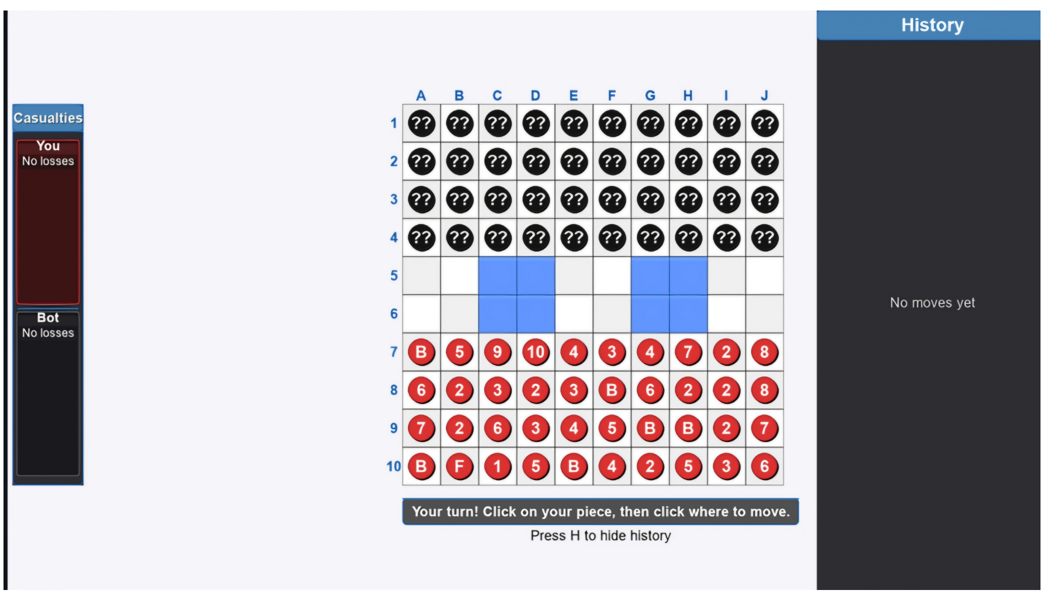

Figure B.3: Game Start

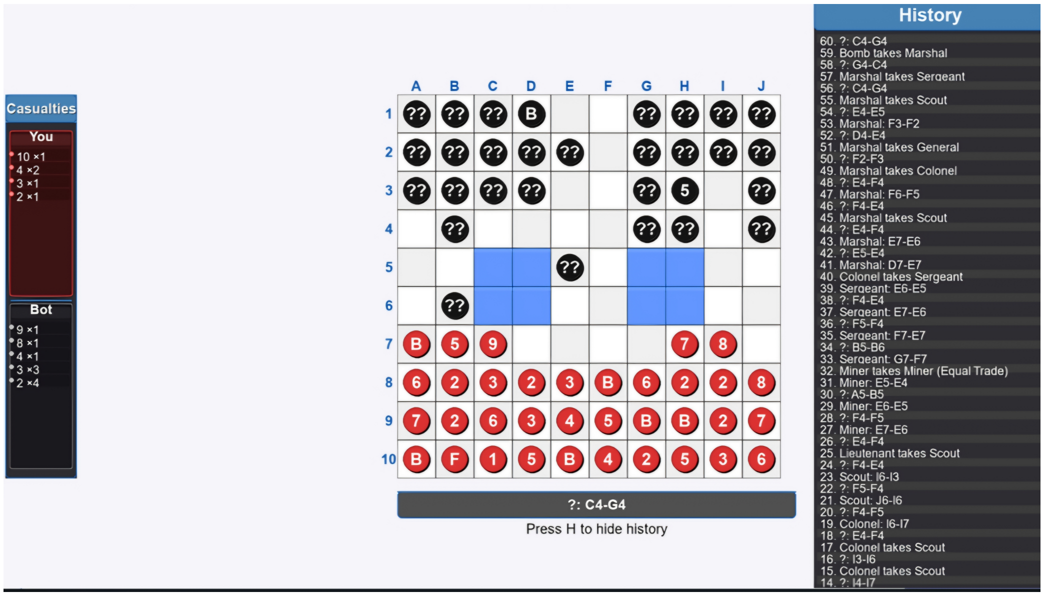

Figure B.4: Battle and History

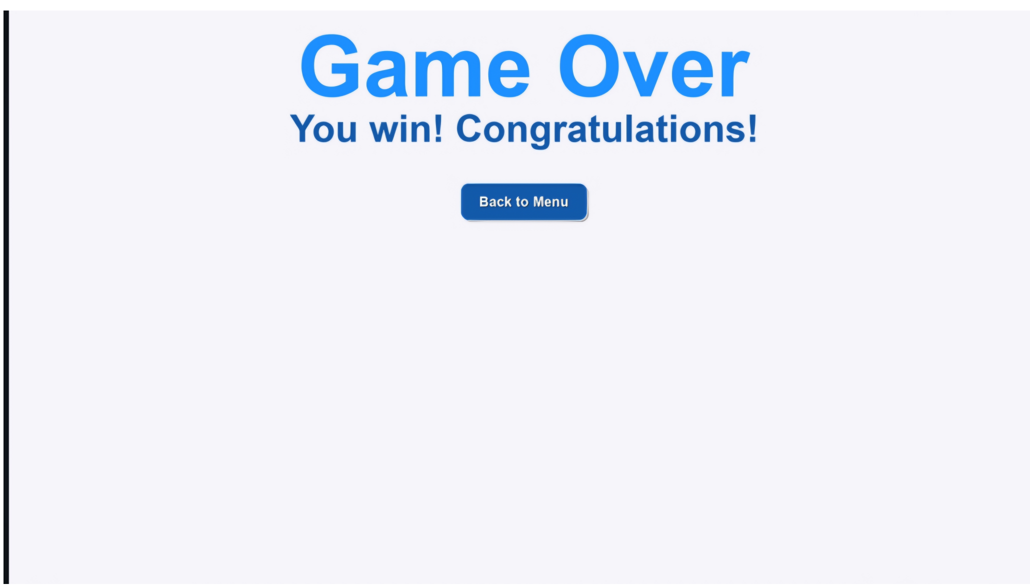

Figure B.5: Victory Screen

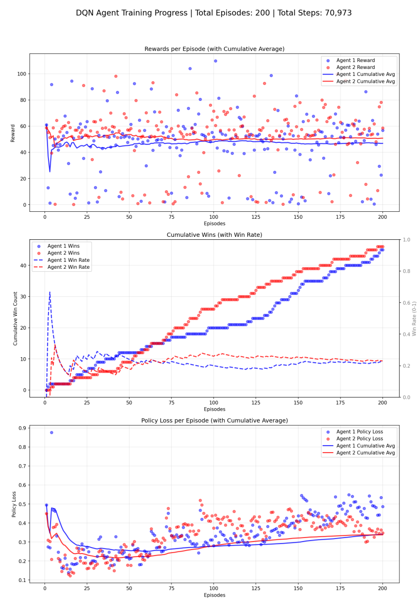

Figure B.6: Training at 200 Episodes

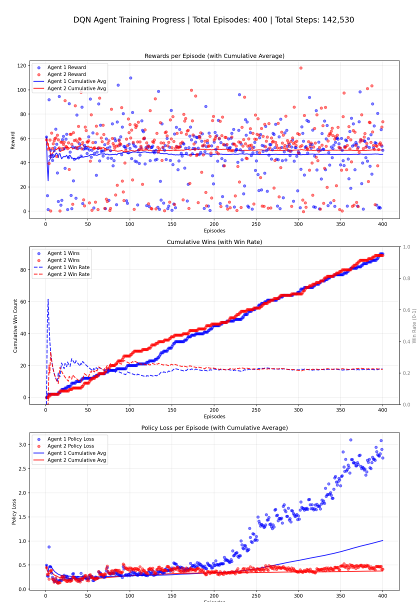

Figure B.7: Training at 400 Episodes

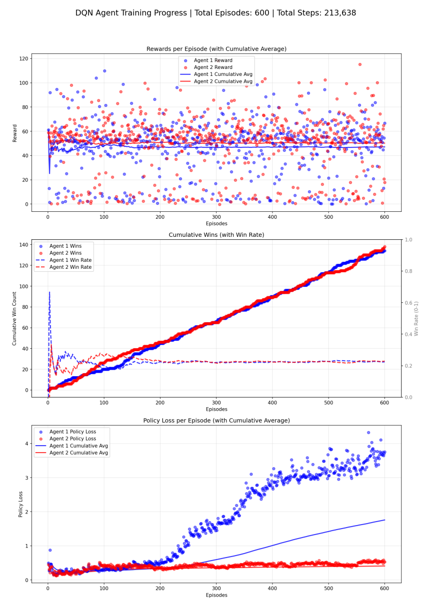

Figure B.8: Training at 600 Episodes

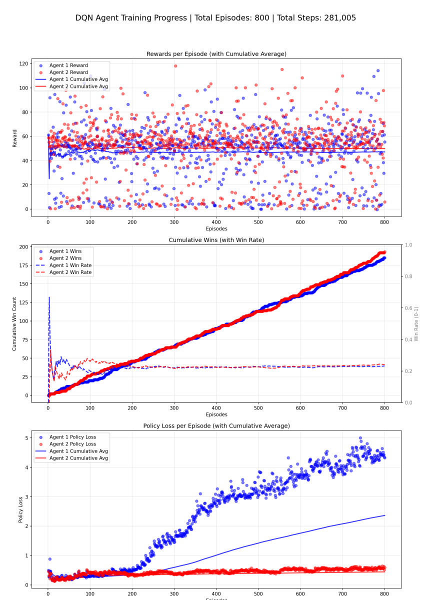

Figure B.9: Training at 800 Episodes

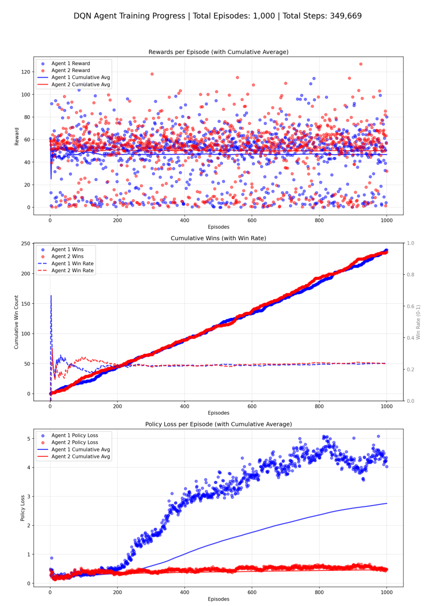

Figure B.10: Training at 1000 Episodes

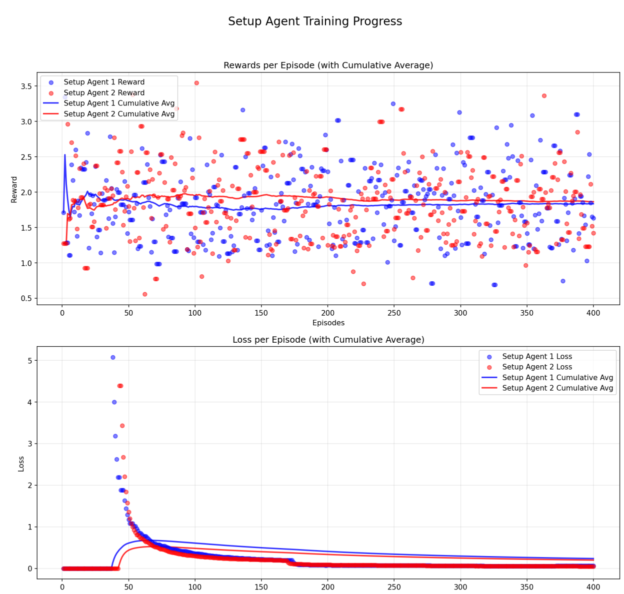

Figure B.11: Setup Agent Training

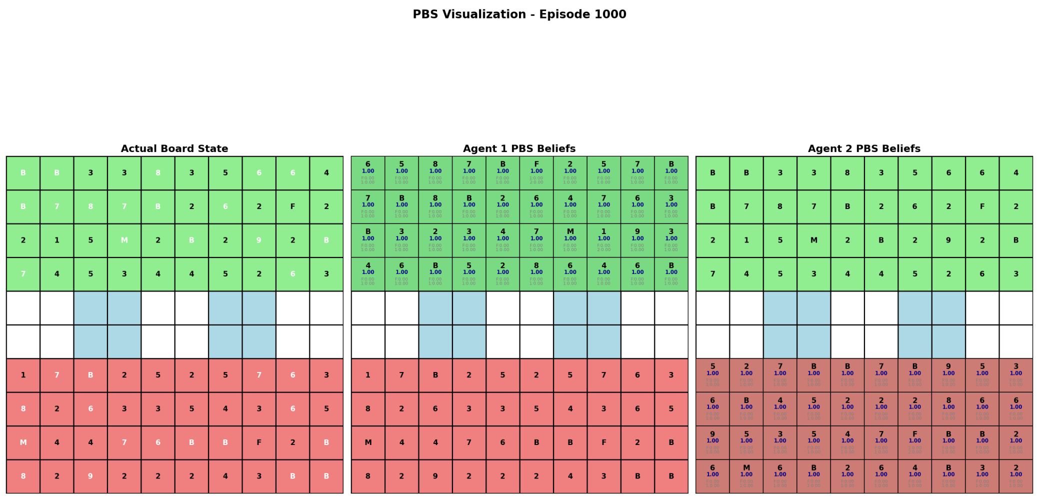

Figure B.12: PBS Visualization

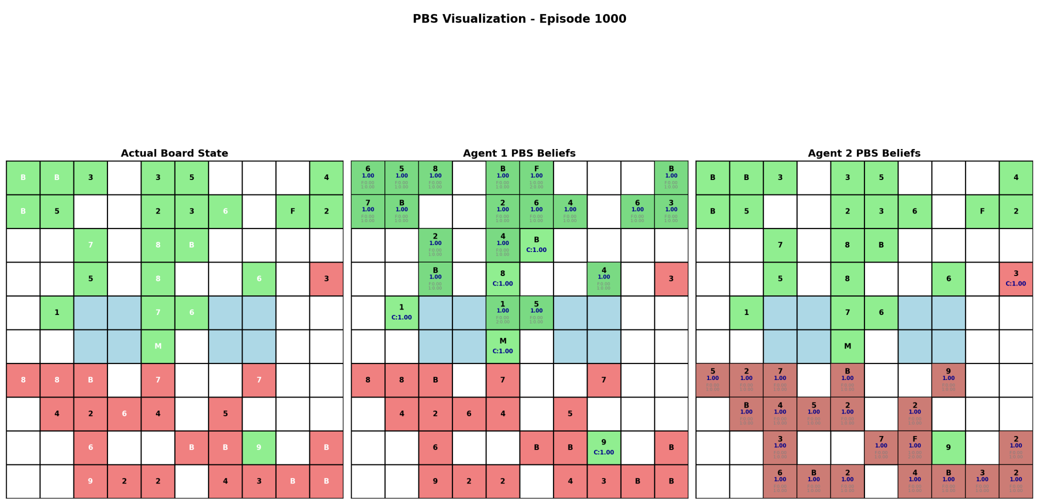

Figure B.13: PBS Visualization

REFERENCES

[1] Adams, G., Padmanabhan, S., and Shekhar, S. Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory. *arXiv preprint arXiv:2012.09136* (2023).

[2] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., and Ragab, M. G. Rnn-lstm: From applications to modeling techniques and beyond—systematic review. *Journal of King Saud University - Computer and Information Sciences 36*, 5 (2024), 102068.

[3] Amaliani, R. N., Soewardikoen, D. W., Azhar, H., and Rahman, Y. Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care. *Advances in Social Humanities Research 3*, 4 (2025).

[4] Arts, A. F. C. Competitive play in stratego. Master's thesis, Maastricht University, Department of Knowledge Engineering, Maastricht, The Netherlands, March 2010. Thesis DKE 10-05.

[5] Barzilai, G., Shamir, O., and Zamani, M. Are convex optimization curves convex? *arXiv preprint arXiv:2503.10138* (2025).

[6] Becker, T., and Sunberg, Z. Imperfect information games and counterfactual regret minimization in space domain awareness. In *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)* (2022), AMOS Technologies.

[7] Bellemare, M. G., Dabney, W., and Rowland, M. *Distributional Reinforcement Learning.* MIT Press, 2023.

[8] Blüml, J., Czech, J., and Kersting, K. Alphaze**: Alphazero-like baselines for imperfect information games are surprisingly strong. *Frontiers in Artificial Intelligence Volume 6 - 2023* (2023).

[9] Brim, A. Deep reinforcement learning pairs trading with a double deep q-network. In *Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, Nevada, USA, Jan.–2020 2020), pp. 222–227.

[10] Brown, N., Lerer, A., Gross, S., and Sandholm, T. Deep counterfactual regret minimization. In *International conference on machine learning* (2019), PMLR, pp. 793–802.

[11] Brown, N., and Sandholm, T. Solving imperfect-information games via subgame solving and depth-limited search. *Artificial Intelligence 297* (2021), 103503.

[12] Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., and Spanò, S. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences 11*, 11 (2021), 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[13] Chen, S., Fried, D., and Topcu, U. Human-agent cooperation in games under incomplete information through natural language communication, 2024.

[14] Chitayat, P. P., Colman, A. M., Hyde, R. J., Drachen, A., and Cowling, P. Wards: Modelling the worth of vision in MOBA's. In *Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys)* (2020), vol. 1251 of *Advances in Intelligent Systems and Computing*, Springer, pp. 55–76.

[15] Dbouk, H. M., Ghorayeb, K., Kassem, H., Hayek, H., Torrens, R., and Wells, O. Facility placement layout optimization. *Journal of Petroleum Science and Engineering 207* (2021), 109079.

[16] De Schutter, B., Babuska, R., and Busoniu, L. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews 38*, 2 (2008), 156–172.

[17] Devlin, S., and Kudenko, D. Potential-based reward shaping in large-scale mdps. *Reinforcement Learning Journal 3*, 2 (2022), 145–167.

[18] El Shar, I., and Jiang, D. Weakly coupled deep q-networks. In *Advances in Neural Information Processing Systems 36* (2023), Curran Associates, Inc. NeurIPS 2023 Conference Track.

[19] Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., and Mori, G. Attention as an rnn. *arXiv preprint arXiv:2405.13956* (2024).

[20] Harrington, J. Let's meetup? board game communities in hong kong. *Games and Culture 20*, 3 (2025), 279–297.

[21] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. *Journal of Machine Learning Research 22*, 85 (2021), 1–52.

[22] Hu, C., Zhao, Y., Wang, Z., Du, H., and Liu, J. Games for artificial intelligence research: A review and perspectives. *IEEE Transactions on Artificial Intelligence 5*, 12 (2024), 5949–5968.

[23] Huang, Y. Deep q-networks. In *Deep Reinforcement Learning: Fundamentals, Research, and Applications*, S. Z. Hao Dong, Zihan Ding, Ed. Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[24] Huang, Y. *Deep Q-Networks*. Springer Singapore, Singapore, 2020, pp. 135–160.

[25] Hugi, D., and Izy, E. Multi-agent deep reinforcement learning (marl) in starcraft 2. `https://crml.eelabs.technion.ac.il/projects/multi-agent-deep-reinforcement-learning-marl-in-starcraft-2/`, 2017. Course project, Winter semester.

[26] Kaur, A., Kumar, K., Prakash, A., and Tripathi, R. Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework. *IEEE Transactions on Cognitive Communications and Networking 9*, 5 (2023), 1271–1281.

[27] Kensert, A., Libin, P., Desmet, G., and Cabooter, D. Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography. *Journal of Chromatography A 1720* (2024), 464768.

[28] Koudela, P. Game of incomplete information and national security. *Central European Political Science Review (CEPSR) 20*, 78 (2019), 73–96.

[29] Le, N., Rathour, V. S., Yamazaki, K., Luu, K., and Savvides, M. Deep reinforcement learning in computer vision: A comprehensive survey, 2021.

[30] Li, B., Fang, Z., and Huang, L. Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning. *arXiv preprint arXiv:2403.04344* (2024).

[31] Li, S., Zhang, Y., Wang, X., Xue, W., and An, B. Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)* (2021), IJCAI, pp. 3663–3669.

[32] Loshchilov, I., and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

[33] Luo, Z., Chen, Z., and Welsh, J. Multi-agent reinforcement learning with deep networks for diverse q-vectors. *arXiv preprint arXiv:2406.07848v1* (2024). Version v1, June 2024.

[34] Nair, R. R., Babu, T., Kishore, S., Pavithra, K., and Sumana, S. G. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)* (Manama, Bahrain, 2025), IEEE, pp. 1–6.

[35] Ouyang, X., and Zhou, T. Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network. *Electronics 12*, 11 (2023), 2453.

[36] Perolat, J., De Vylder, B., Hennes, D., Tarassov, E., Strub, F., de Boer, V., Muller, P., Connor, J. T., Burch, N., Anthony, T., et al. Mastering the game of stratego with model-free multiagent reinforcement learning. *Science 378*, 6623 (2022), 990–996.

[37] Petersen, C. The reality of the board game community: And the connection, identity and experience that creates it. Master's thesis, Eastern University, Aug. 2024.

[38] Qu, T., Zhou, Q., Zhu, J., and Duan, F. Strategy optimization of imperfect information games based on nfsp with ddqn. In *Advances in Guidance, Navigation and Control*, L. Yan, H. Duan, and Y. Deng, Eds., vol. 845 of *Lecture Notes in Electrical Engineering*. Springer, Singapore, 2023, pp. 4376–4383.

[39] Saffidine, A., Finnsson, H., and Buro, M. Alpha-beta pruning for games with simultaneous moves. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 26, pp. 556–562.

[40] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[41] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., Davoodi, E., Christianson, A., and Bowling, M. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[42] Schofield, M., and Thielscher, M. General game playing with imperfect information. *Journal of Artificial Intelligence Research 66* (2019), 901–935.

[43] Sokota, S., Vinitsky, E., Hu, H., Kolter, J. Z., and Farina, G. Superhuman ai for stratego using self-play reinforcement learning and test-time search, 2025.

[44] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature 575*, 7782 (2019), 350–354. Technical supplement updated 2021.

[45] Wang, X., Li, Y., and Zhang, W. Deep reinforcement learning in imperfect-information games: A survey. *IEEE Transactions on Games 15*, 3 (2023), 421–438.

[46] Wang, Y. Game-theoretic understandings of multi-agent systems with multiple objectives, 2025.

[47] Wang, Y., and Wellman, M. P. Regularization for strategy exploration in empirical game-theoretic analysis, 2023.

[48] Wongkamjan, W., Gu, F., Wang, Y., Hermjakob, U., May, J., Stewart, B., Kummerfeld, J., Peskoff, D., and Boyd-Graber, J. More victories, less cooperation: Assessing cicero's diplomacy play. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, p. 12423–12441.

[49] Xu, H., Li, K., Fu, H., Fu, Q., and Xing, J. Autocfr: Learning to design counterfactual regret minimization algorithms. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)* (2022), Association for the Advancement of Artificial Intelligence.

[50] Xu, H., Li, K., Fu, H., FU, Q., Xing, J., and Cheng, J. Dynamic discounted counterfactual regret minimization. In *The Twelfth International Conference on Learning Representations* (2024).

[51] Zakharenkov, A., and Makarov, I. Deep reinforcement learning with dqn vs. ppo in vizdoom. In *2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI)* (Budapest, Hungary, 2021), IEEE, pp. 131–136.

[52] Zhang, B., and Sandholm, T. Subgame solving without common knowledge. *Advances in Neural Information Processing Systems 34* (2021), 23993–24004.

[53] Zhang, B. H., and Sandholm, T. General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess. *arXiv preprint arXiv:2506.01242* (2025).

[54] Zhang, K., Yang, Z., and Başar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[55] Zheng, S., Trott, A., Srinivasa, S., Naik, N., Gruesbeck, M., Parkes, D. C., and Socher, R. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332* (2020).

[56] Zhu, K., Liu, Q., Yan, X., Wu, F., Zhao, X., Zhou, P., and Yang, X. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems 34*, 7 (2022), 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.